データモデル

エンティティ一覧

Otiumアプリケーションは以下のエンティティで構成されています：

1. **users** - ユーザー情報
2. **records** - 記録
3. **categories** - カテゴリー
4. **user_avatars** - ユーザーアバター画像
5. **follows** - フォロー関係

ER図

```
┌──────────────────────────────────────────────────────────────────────────────┐
│           users                    │
├──────────────────────────────────────────────────────────────────────────────┤
│ * id (PK)                          │
│   email (UNIQUE)                   │
│   user_name                        │
│   password_hash                    │
│   bio                              │
│   visibility                       │
│   search_key                       │
│   default_view_mode                │
│   default_sort_order               │
│   created_at                       │
│   updated_at                       │
└──────────────────────────────────────────────────────────────────────────────┘
                          │
              │ 1
              │
     ┌────────────────┼────────────────┐
     │         │         │ N
     │ N       │         │
┌────────▼────────┐  ┌──▼──────────────────────┐  ┌────────▼────────────────┐
│ follows │  │    records      │  │  categories  │
├─────────────────┤  ├─────────────────────────┤  ├─────────────────────────┤
│ * id    │  │ * id (PK)       │  │ * id (PK)   │
│ follower│  │   user_id (FK)  │  │   user_id   │
│ _id(FK) │  │   title         │  │   name      │
│ follow  │  │   description   │  │   created_at│
│ ing_id  │  │   date_logged   │  │   updated_at│
│ (FK)    │  │   image_url     │  └─────────────────────────┘
└─────────────────┘  │   category_id   │
```

```
            │   show_in_timeline│
            │   invalidation   │
            │   _flag          │
            │   created_at     │
            │   updated_at     │
            │   delete_at      │
            └──────────────────────────────┘


┌──────────────────────────────────────────────────────────────────────────┐
│        user_avatars                 │
├──────────────────────────────────────────────────────────────────────────┤
│ * id (PK)                           │
│   user_id (FK) → users.id            │
│   image_url                         │
│   created_at                        │
│   updated_at                        │
└──────────────────────────────────────────────────────────────────────────┘
```

エンティティ詳細

1. users（ユーザー）

ユーザーアカウント情報を管理するテーブル

カラム名	データ型	制約	説明
id	INT	PRIMARY KEY, AUTO_INCREMENT	ユーザーID（主キー）
email	VARCHAR(255)	UNIQUE, NOT NULL	メールアドレス（ログインID）
user_name	VARCHAR(255)	NULL	ユーザー表示名
password_hash	VARCHAR(255)	NOT NULL	bcryptでハッシュ化されたパスワード
bio	TEXT	NULL	自己紹介
visibility	VARCHAR(20)	DEFAULT 'private'	公開設定（`public` | `private`）
search_key	VARCHAR(100)	NULL, UNIQUE	非公開時の検索キー（例: "123_abc123...")
default_view_mode	VARCHAR(20)	DEFAULT 'grid'	一覧のデフォルト表示形式（`grid` |

カラム名	データ型	制約	説明
			`list` | `booklist` | `tile`)
default_sort_order	VARCHAR(20)	DEFAULT 'date_logged'	一覧のデフォルト並び順 (`date_logged` | `created_at`)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	作成日時
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	更新日時

インデックス:

- PRIMARY KEY: `id`
- UNIQUE INDEX: `email`
- UNIQUE INDEX: `search_key`

関連:

- 1対多: records（1ユーザーは複数の記録を持つ）
- 1対多: categories（1ユーザーは複数のカテゴリーを持つ）
- 1対1: user_avatars（1ユーザーは1つのアバター画像を持つ）
- 多対多: follows（フォロー関係）

---

2. records（記録）

ユーザーの記録データを管理するテーブル

カラム名	データ型	制約	説明
id	INT	PRIMARY KEY, AUTO_INCREMENT	記録ID（主キー）
user_id	INT	FOREIGN KEY, NOT NULL	ユーザーID（外部キー → users.id）
title	VARCHAR(255)	NOT NULL	記録のタイトル（デフォルト: "無題の記録"）
description	TEXT	NULL	記録の詳細説明
date_logged	DATE	NOT NULL	記録日付

カラム名	データ型	制約	説明
image_url	VARCHAR(500)	NULL	画像ファイルのパス（例: "uploads/filename.jpg"）
category_id	INT	FOREIGN KEY, NULL	カテゴリーID（外部キー → categories.id）
show_in_timeline	TINYINT(1)	DEFAULT 1	スレッドタイムラインに表示するか（0: 非表示, 1: 表示）
invalidation_flag	TINYINT(1)	DEFAULT 0	削除フラグ（0: 有効, 1: 削除済み）
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	作成日時
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	更新日時
delete_at	TIMESTAMP	NULL	削除日時（論理削除時）

インデックス:

- PRIMARY KEY: `id`
- FOREIGN KEY: `user_id` REFERENCES `users(id)`
- FOREIGN KEY: `category_id` REFERENCES `categories(id)`
- INDEX: `user_id, invalidation_flag`（検索性能向上のため）

関連:

- 多対1: users（複数の記録は1ユーザーに属する）
- 多対1: categories（複数の記録は1カテゴリーに属する、NULL可）

ビジネスルール:

- `invalidation_flag=1` の記録は表示されない（論理削除）
- `category_id` が NULL の場合はカテゴリー未設定
- `date_logged` は必須
- `show_in_timeline=1` の記録のみ、フォロー中のユーザーのタイムラインに表示される

---

3. categories（カテゴリー）

記録を分類するためのカテゴリーを管理するテーブル

カラム名	データ型	制約	説明
id	INT	PRIMARY KEY, AUTO_INCREMENT	カテゴリーID（主キー）
user_id	INT	FOREIGN KEY, NOT NULL	ユーザーID（外部キー → users.id）
name	VARCHAR(255)	NOT NULL	カテゴリー名
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	作成日時
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	更新日時

インデックス:

- PRIMARY KEY: `id`
- FOREIGN KEY: `user_id` REFERENCES `users(id)`

関連:

- 多対1: users（複数のカテゴリーは1ユーザーに属する）
- 1対多: records（1カテゴリーは複数の記録を持つ）

ビジネスルール:

- ユーザーごとに独立したカテゴリー
- `name` は必須

注意: アイコン・色設定は削除され、APIでは `name` のみ使用します。

---

5. user_avatars（ユーザーアバター画像）

ユーザーのアバター画像を管理するテーブル

カラム名	データ型	制約	説明
id	INT	PRIMARY KEY, AUTO_INCREMENT	アバターID（主キー）
user_id	INT	FOREIGN KEY, NOT NULL, UNIQUE	ユーザーID（外部キー → users.id）
image_url	VARCHAR(500)	NOT NULL	画像ファイルのパス（例: "uploads/avatar-filename.jpg"）

カラム名	データ型	制約	説明
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	作成日時
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	更新日時

インデックス:

- PRIMARY KEY: `id`
- FOREIGN KEY: `user_id` REFERENCES `users(id)`
- UNIQUE INDEX: `user_id`（1ユーザーに1アバターのみ）

関連:

- 1対1: users（1アバターは1ユーザーに属する）

ビジネスルール:

- 1ユーザーにつき1アバター画像のみ設定可能
- アバター画像は任意（設定されていない場合もある）

---

6. follows（フォロー関係）

ユーザー間のフォロー関係を管理するテーブル

カラム名	データ型	制約	説明
id	INT	PRIMARY KEY, AUTO_INCREMENT	フォローID（主キー）
follower_id	INT	FOREIGN KEY, NOT NULL	フォローするユーザーID（外部キー → users.id）
following_id	INT	FOREIGN KEY, NOT NULL	フォローされるユーザーID（外部キー → users.id）
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	作成日時

インデックス:

- PRIMARY KEY: `id`
- UNIQUE KEY: `(follower_id, following_id)`（重複フォロー防止）
- FOREIGN KEY: `follower_id` REFERENCES `users(id)` ON DELETE CASCADE
- FOREIGN KEY: `following_id` REFERENCES `users(id)` ON DELETE CASCADE

- INDEX: `follower_id`、`following_id`

制約:

- 自分自身はフォローできない（`follower_id != following_id`）

関連:

- 多対1: users（follower として）
- 多対1: users（following として）

データ型の詳細

文字列型

- **VARCHAR(n)**: 可変長文字列（最大n文字）
- **TEXT**: 長文テキスト（最大65,535文字）

数値型

- **INT**: 32ビット整数（-2,147,483,648 ～ 2,147,483,647）
- **TINYINT(1)**: 1バイト整数（0 ～ 255、フラグとして使用）
- **DECIMAL(p,s)**: 固定精度の小数（p: 全体の桁数、s: 小数部の桁数）

日時型

- **DATE**: 日付（YYYY-MM-DD）
- **TIMESTAMP**: 日時（YYYY-MM-DD HH:MM:SS）

データ整合性ルール

外部キー制約

1. **records.user_id → users.id**
   - ユーザーが削除された場合の動作: 未定義（現状、ユーザー削除機能なし）
2. **records.category_id → categories.id**
   - カテゴリーが削除された場合: `category_id` は NULL になる（物理削除のため）
3. **user_avatars.user_id → users.id**
   - ユーザーが削除された場合の動作: 未定義（現状、ユーザー削除機能なし）

論理削除

- **records** テーブルは論理削除を採用
  - `invalidation_flag=1` で削除済みを表現
  - `delete_at` に削除日時を記録
  - 物理削除は行わない（データの復元が可能）

- **categories** テーブルは物理削除を採用
  - 削除時はレコードを完全に削除
  - 関連する `records.category_id` は NULL になる

データベース設計の考慮事項

パフォーマンス

1. **インデックス**
   - `users.email`：ログイン時の検索を高速化
   - `records(user_id, invalidation_flag)`：ユーザーの記録一覧取得を高速化
2. **文字コード**
   - UTF-8（utf8mb4）を使用し、絵文字なども保存可能

セキュリティ

1. **パスワード**
   - `password_hash` に平文パスワードは保存しない
   - bcryptでハッシュ化（ソルトラウンド10）
2. **データ分離**
   - すべてのクエリで `user_id` によるフィルタリングを実施
   - ユーザー間でデータが混在しないようにする

拡張性

1. **画像URL**
   - `image_url` は相対パスで保存（例: "uploads/filename.jpg"）
   - 将来的にCDNやS3に移行しやすい設計
2. **カテゴリー**
   - ユーザーごとに独立したカテゴリー
   - 将来的に共有カテゴリー機能を追加可能

データベース初期化

テーブル作成・マイグレーションスクリプトは `server-api/scripts/` ディレクトリにあります：

- `create_categories_table.js` - categories テーブル作成
- `create_user_avatars_table.js` - user_avatars テーブル作成
- `create_follows_table.js` - follows テーブル作成
- `add_bio_column_to_users.js` - users に bio カラム追加

- `add_visibility_columns_to_users.js` - users に visibility, search_key 追加
- `add_default_view_mode_to_users.js` - users に default_view_mode 追加
- `add_default_sort_order_to_users.js` - users に default_sort_order 追加
- `add_show_in_timeline_to_records.js` - records に show_in_timeline 追加

データベース設計ファイル（A5:SQL Mk-2形式）は `Doc/DDL/otium.a5er` にあります。